

Salesforce Assignment 19: Product Inventory Management System (Salesforce Apex + Visualforce)

Step 1: Create Custom Object

Go to: Setup → Object Manager → Create → Custom Object

Create object:

- **Label:** Product Inventory
- **Plural Label:** Product Inventories
- **Object Name:** Product_Inventory
- **Record Name:** Product Name
- **Data Type:** Text

Click **Save**.

Step 2: Create Custom Fields

Inside **Product Inventory** object, create these fields:

1. Serial No

- Data Type: Number
- Field Label: Serial No
- Length: 10
- Decimal Places: 0
- API Name:

2. Manufacture Date

- Data Type: Date
- API Name:

3. Expiry Date

- Data Type: Date
- API Name:

Default Salesforce field:

- Product Name →

Step 3: Create Apex Controller Class

Go to: Setup → Apex Classes → New

Class Name: ProductInventoryController

Paste this code:

```
public class ProductInventoryController {

    public Product_Inventory__c newProd {get; set;}

    public ProductInventoryController() {
        newProd = new Product_Inventory__c();
    }

    public PageReference saveProduct() {
        try {

            // Validation
            if(newProd.Expiry_Date__c <= newProd.Manufacture_Date__c) {
                ApexPages.addMessage(
                    new ApexPages.Message(
                        ApexPages.Severity.ERROR,
                        'Expiry Date must be greater than Manufacture Date'
                    )
                );
                return null;
            }

            // Insert record
            insert newProd;

            // Clear form
            newProd = new Product_Inventory__c();

            ApexPages.addMessage(
                new ApexPages.Message(
                    ApexPages.Severity.CONFIRM,
                    'Product Saved Successfully'
                )
            );

            return null;

        } catch(Exception e) {
            ApexPages.addMessages(e);
            return null;
        }
    }
}
```

```

public List<Product_Inventory__c> getInventoryList() {
    return [
        SELECT Name,
            Serial_No__c,
            Manufacture_Date__c,
            Expiry_Date__c
        FROM Product_Inventory__c
        ORDER BY CreatedDate DESC
    ];
}
}

```

Click **Save**.

Step 4: Create Visualforce Page

Go to: Setup → Visualforce Pages → New

Page Name:

Paste this code:

```

<apex:page controller="ProductInventoryController">
    <apex:form id="inventoryForm">

        <!-- Shows validation/success messages -->
        <apex:pageMessages id="msg"/>

        <!-- Input Form -->
        <apex:pageBlock title="Add New Product Inventory">
            <apex:pageBlockSection columns="1">
                <apex:inputField value="{!newProd.Name}" />
                <apex:inputField value="{!newProd.Serial_No__c}" />
                <apex:inputField value="{!newProd.Manufacture_Date__c}" />
                <apex:inputField value="{!newProd.Expiry_Date__c}" />
            </apex:pageBlockSection>

            <apex:pageBlockButtons>
                <apex:commandButton
                    value="Save Inventory"
                    action="{!saveProduct}"
                    reRender="inventoryForm,msg" />
            </apex:pageBlockButtons>
        </apex:pageBlock>

        <!-- Inventory Table -->
        <apex:pageBlock title="Current Inventory List">
            <apex:pageBlockTable value="{!inventoryList}" var="item">
                <apex:column value="{!item.Name}" headerValue="Product Name"/
            >

```

```

                <apex:column value="{!item.Serial_No__c}"
headerValue="Serial Number"/>
                <apex:column value="{!item.Manufacture_Date__c}"
headerValue="Mfg Date"/>
                <apex:column value="{!item.Expiry_Date__c}"
headerValue="Expiry Date"/>
            </apex:pageBlockTable>
        </apex:pageBlock>

    </apex:form>
</apex:page>

```

Click **Save**.

Step 5: Run Application

- Open Visualforce page preview
- Enter Product Name
- Enter Serial No
- Enter Manufacture Date
- Enter Expiry Date
- Click **Save Product**
- Products will display in the table
- To remove a product, click **Delete**

Features Implemented

- Add Product
- View Products
- Delete Product
- Frontend using Visualforce
- Backend using Apex
- Custom object storage

Viva Points

- `Product_Inventory__c` is a custom object
- `Name` is Salesforce default record name field
- Apex controller handles backend logic
- Visualforce provides frontend UI
- SOQL fetches product records
- DML operations used: `insert` and `delete`